SOEN 6011 Deliverable 2: F6 Beta Function

From-Scratch Numerical Core, Tkinter Graphical Interface, Version Control, and Updated Requirements

Wei Huang

Concordia University

Summer 2026

Deliverable 2 Scope and Dependency Chain

Deliverable 1 baseline

- Log-gamma identity selected
- Python textual user interface
- `math.lgamma` and `math.exp`

Deliverable 2 increment

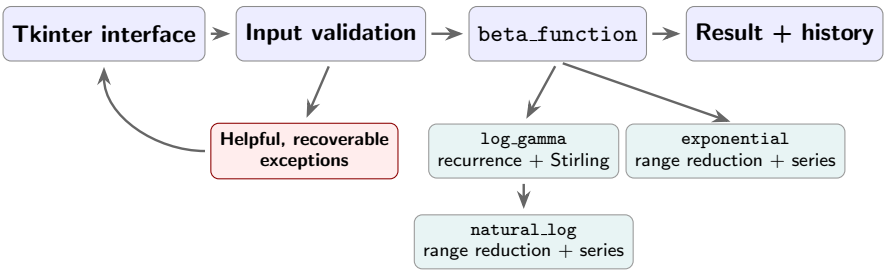
1. **Problem 5:** implement mathematics from scratch
2. **Problem 5:** build a Tkinter graphical interface
3. **Problem 6:** publish Git repository + README
4. **Problem 7:** revise requirements

Interpretation of "from scratch"

The numerical core uses no built-in or library functions except those permitted for input, output, arithmetic, user-interface design, and exception handling. (Kamthan, 2026a)

Deliverable 2 revisits the Deliverable 1 solution and adds new behavior, following an iterative and incremental process. (Kamthan, 2026b)

Problem 5: From-Scratch Architecture



Subordinate functions implemented from scratch: `natural_log`, `exponential`, and `log_gamma`.

Allowed dependencies

`tkinter`, `ttk`, float parsing, formatting, arithmetic.

Prohibited dependencies avoided

No other built-in or library functions are used by the numerical core.

Problem 5: Numerical Method and Verification

Subordinate mathematics

$$\ln m = 2\sum_{k=0}^{\infty} \frac{z^{2k+1}}{2k+1}, \quad z = \frac{m-1}{m+1}, \; 1 \le m < 2$$

$$e^r = \sum_{k=0}^{\infty} \frac{r^k}{k!}$$

Shift x to $x \ge 8$, then use the Gamma recurrence and a Stirling correction series.

$$\log B = \log \Gamma(x) + \log \Gamma(y) - \log \Gamma(x+y)$$

External verification only

Input	Computed value	Rel. error
$B(1,1)$	0.999999999999988	1.24×10^{-14}
$B(2,3)$	0.0833333333333322	1.45×10^{-14}
$B(0.5,0.5)$	3.14159265358979	1.70×10^{-15}
$B(100,300)$	$5.94745626861 \times 10^{-99}$	9.40×10^{-13}

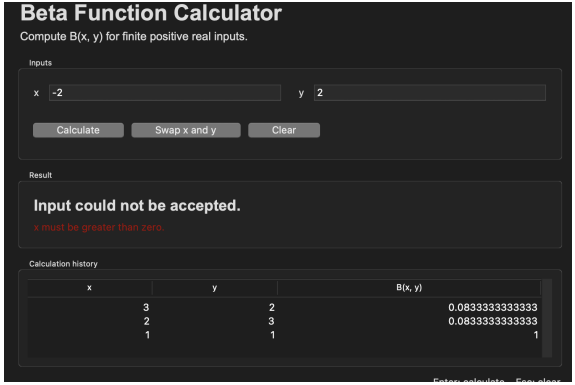
Numeric range and measured timing

Underflow reports that the positive result is smaller than the minimum nonzero Python float; zero is not returned.

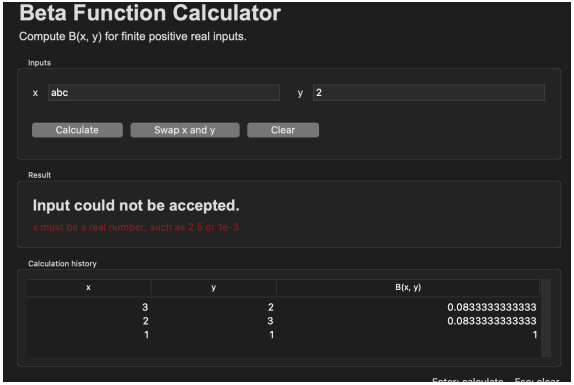
Benchmark: 1,000 calls for each of five documented inputs; maximum observed time = 0.129 ms on an Apple M4 MacBook Pro with Python 3.13.0.

All verification values were produced externally and used only for validation. The submitted implementation itself imports no mathematical library. (NIST DLMF, 2026a; NIST DLMF, 2026b)

Problem 5: Tkinter Interface and Exception Recovery



Domain error: names x and states the $x > 0$ correction.

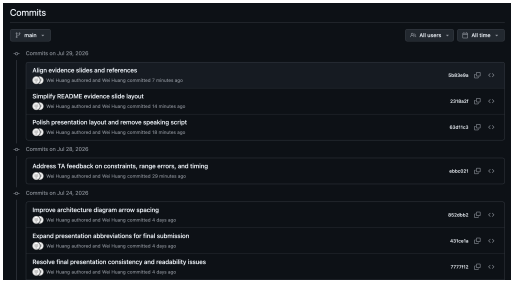


Nonnumeric error: names x and provides valid examples.

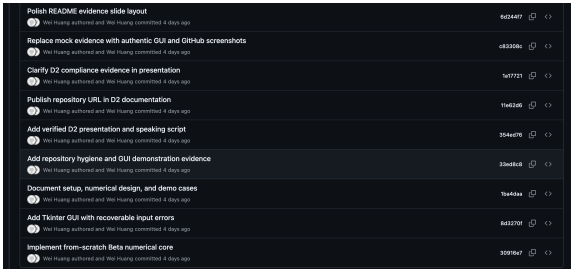
Implementation and recovery evidence

Graphical interface implemented entirely with Tkinter widgets. Both screenshots show that handled errors preserve calculation history and leave the window ready for corrected input.

Problem 6: Distributed Version Control Evidence



Recent refinement and publication commits



Core, interface, documentation, and evidence commits

What the history demonstrates

Purpose-focused commits separate the numerical core, Tkinter recovery, documentation, evidence, and final delivery. Commit logs make each increment explicit. (Kamthan, 2026c)

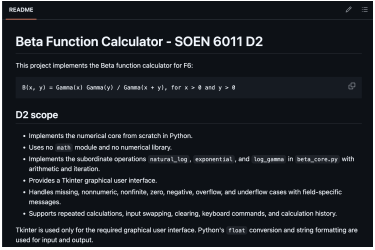
Public repository

GitHub Repository

`github.com/HW925/SOEN-6011-D2-Beta-Function`

Problem 6 – README Evidence

Scope and boundary



README

Beta Function Calculator - SOEN 6011 D2

This project implements the Beta function calculator for F6:

```
B(x, y) = Gamma(x) Gamma(y) / Gamma(x + y), for x > 0 and y > 0
```

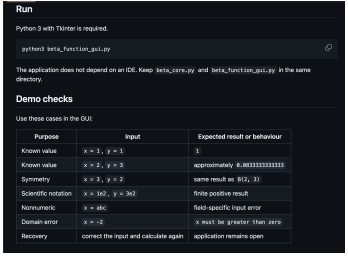
D2 scope

- Implements the numerical core from scratch in Python.
- Uses no `math` module and no numerical library.
- Implements the subordinate operations `natural_log`, `exponential`, and `log_gamma` in `beta_core.py` with arithmetic and iteration.
- Provides a Tkinter graphical user interface.
- Handles missing, nonnumeric, nonfinite, zero, negative, overflow, and underflow cases with field-specific messages.
- Supports repeated calculations, input swapping, clearing, keyboard commands, and calculation history.

Tkinter is used only for the required graphical user interface. Python's `float` conversion and string formatting are used for input and output.

From-scratch functions, permitted operations, Tkinter, and handled cases.

Run and verify



Run

Python 3 with Tkinter is required.

```
python3 beta_function_gui.py
```

The application does not depend on an IDE. Keep `beta_core.py` and `beta_function_gui.py` in the same directory.

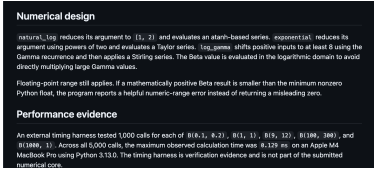
Demo checks

Use these cases in the GUI:

Purpose	Input	Expected result or behaviour
Known value	`x = 1`, `y = 1`	`1`
Known value	`x = 2`, `y = 3`	approximately `0.0833333333333`
Symmetry	`x = 3`, `y = 2`	same result as `B(2, 3)`
Scientific notation	`x = 1e2`, `y = 3e2`	finite positive result
Nonnumeric	`x = abc`	field-specific input error
Domain error	`x = -2`	`x must be greater than zero`
Recovery	correct the input and calculate again	application remains open

Terminal command and reproducible demonstration checks.

Design and repository



Numerical design

`natural_log` reduces its argument to `[1, 2)` and evaluates an atanh-based series. `exponential` reduces its argument using powers of two and evaluates a Taylor series. `log_gamma` shifts positive inputs to at least 8 using the Gamma recurrence and then applies a Stirling series. The Beta value is evaluated in the logarithmic domain to avoid directly multiplying large Gamma values.

Floating-point range still applies. If a mathematically positive Beta result is smaller than the minimum nonzero Python float, the program reports a helpful numeric-range error instead of returning a misleading zero.

Performance evidence

An external timing harness tested 1,000 calls for each of `B(0.5, 0.2)`, `B(1, 1)`, `B(9, 12)`, `B(100, 200)`, and `B(1000, 1)`. Across all 5,000 calls, the maximum observed calculation time was `0.129 ms` on an Apple M4 MacBook Pro using Python 3.13.0. The timing harness is verification evidence and is not part of the submitted numerical core.

Numerical method, floating-point limits, and public repository.

The README connects implementation scope, reproducible use, numerical design, and repository access.

Problem 7: Updated Functional Requirements

Legend: FR denotes Functional Requirement. These replace the Deliverable 1 textual-interface requirements.

Graphical-interface interaction and calculation

D2-FR-01 The system shall provide labeled graphical fields for x and y, plus Calculate, Swap, and Clear commands.

D2-FR-02 The system shall accept finite positive real inputs in decimal or scientific notation.

D2-FR-03 The system shall compute $B(x, y)$ through the from-scratch `natural_log`, `exponential`, and `log_gamma` operations.

D2-FR-04 The system shall display each representable result with at least ten significant digits.

D2-FR-05 The system shall preserve a history of successful calculations during the current session.

D2-FR-06 The system shall allow another calculation after a result or handled error without restarting.

Numeric-range behavior

D2-FR-07 The system shall detect an unrepresentable result and display a helpful numeric-range error instead of returning zero or infinity.

Problem 7: Constraints and Quality

Constraints

D2-CR-01 The graphical interface shall use Tkinter.

D2-CR-02 The numerical core shall not use built-in or library functions except those permitted for input, output, arithmetic, user-interface design, and exception handling.

D2-CR-03 The application shall run from a terminal with `python3 beta_function_gui.py`, independent of an integrated development environment.

D2-CR-04 Each handled input error shall name the field and the correction needed.

D2-CR-05 The system shall catch supported input and numeric-range exceptions without terminating the graphical interface.

Quality requirements

D2-QR-01 Relative error shall be at most 10^{-10} for $B(1,1) = 1$ and $B(2,3) = 1/12$; $B(2,3)$ and $B(3,2)$ shall agree within the same tolerance.

D2-QR-02 For the documented benchmark set, each calculation shall complete within 100 ms on the demonstration computer.

D2-QR-03 After a handled error, the graphical interface shall remain open and ready for corrected input.

Traceability: Problem 5 architecture satisfies D2-CR-01–05; verification supports D2-QR-01–02; range and recovery evidence supports D2-FR-06–07 and D2-QR-03. (ISO/IEC/IEEE, 2018; Kamthan, 2026a)

Generative Artificial Intelligence: CASTROFF Evidence

Tool and prompt structure

Tool: OpenAI ChatGPT/Codex
Context: Deliverable 2, F6
Audience: TA and instructor

Style: concise, evidence-based
Task: Problems 5–7
Role: software engineering student

Output: code, README, requirements, slides
Format: runnable files and LaTeX
Feedback: verify by mathematics and execution

Problem 5

Prompt
Design Beta from scratch, including subordinate functions and exceptions.

Decision
Used recurrence and Stirling; added range checks; rejected direct Gamma multiplication.

Problem 6

Prompt
Propose purpose-focused commits and a minimal README for the graphical interface.

Decision
Accepted after checking repository history and executing README commands locally.

Problem 7

Prompt
Revise Deliverable 1 requirements for the graphical interface and from-scratch constraint.

Decision
Separated functional, constraint, accuracy, timing, range, and recovery requirements.

References

Kamthan, P. (2026a). *SOEN 6011 Project Description*. Concordia University, Summer 2026.

Kamthan, P. (2026b). *Introduction to Software Processes*. SOEN 6011 lecture notes.

Kamthan, P. (2026c). *Introduction to Software Engineering Documentation*. SOEN 6011 lecture notes.

NIST Digital Library of Mathematical Functions. (2026a). Section 5.12, Beta Function, Release 1.2.7.

NIST Digital Library of Mathematical Functions. (2026b). Section 5.2, Gamma Function Definitions, Release 1.2.7.

ISO/IEC/IEEE. (2018). *29148:2018 Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*.